

Práctica 14: Sistema de Cruce Peatonal

Microcontroladores

Alumno: Itzel Estrada Arcos (368980)

Docente: Ing. Jesus Padron

Viernes 2 de mayo 2025

0.1 Introduction.

En este proyecto desarrollamos un sistema de semáforo inteligente utilizando la Raspberry Pi Pico y el lenguaje de programación C. El objetivo principal es simular el funcionamiento coordinado de semáforos vehiculares y peatonales, incorporando control de cruce mediante botones y una cuenta regresiva visualizada en un display doble de 7 segmentos. Para lograr un funcionamiento eficiente y responsivo, aprovechamos ambos núcleos del microcontrolador de la Pico, separando las tareas de monitoreo de entradas y control de la lógica del tráfico. Además, configuramos el entorno de desarrollo mediante CMake para compilar el código y generar el archivo necesario para cargar el programa en la placa.

0.2 Desarrollo de la practica.

En este proyecto implementamos un sistema de semáforo utilizando la Raspberry Pi Pico y el lenguaje C. El código inicia incluyendo las librerías necesarias para controlar los pines GPIO y utilizar ambos núcleos del microcontrolador. Definimos los pines para los semáforos vehiculares, peatonales, botones y un display doble de 7 segmentos de cátodo común. Luego, inicializamos los pines como entradas o salidas según su función y se encienden las luces rojas peatonales por defecto para indicar que no se puede cruzar.

Utilizamos funciones auxiliares para controlar de forma organizada los semáforos y para verificar si los botones peatonales han sido presionados. Estas pulsaciones solo se registran si ocurren mientras el semáforo vehicular correspondiente está en verde. También manejamos una cuenta regresiva en el display durante el cruce peatonal, con parpadeo de la luz roja para advertencia. Al finalizar la cuenta, se restablece la luz roja fija.

El flujo del tráfico lo dividimos en fases, donde una calle recibe luz verde mientras la otra permanece en rojo. Si hay una solicitud peatonal pendiente en la calle con luz roja, se ejecuta. El segundo núcleo de la Pico se encarga exclusivamente de leer los botones sin interrupciones, mientras que el núcleo principal alterna entre fases de manera continua y automática.

0.2.1 Proceso paso a paso.

Creamos una nueva carpeta llamada "Práctica-14". A la que agregaremos los archivos importantes de la aplicación, estos archivos incluyen .vscode, picosdkimport.cmake, CMakeLists.txt y un archivo de código fuente en C. Este último se llamará Semaforo.c.

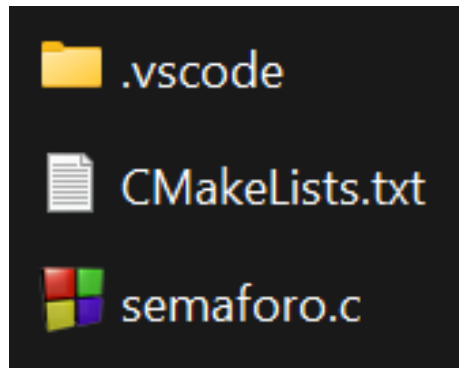


Figure 1: Archivos

Código en C.

Para este código, comenzamos incluyendo las librerías necesarias para trabajar con la Raspberry Pi Pico, especialmente para el manejo de los pines GPIO y para aprovechar el uso de los dos núcleos del procesador. Definimos los números de los pines que utilizamos en el sistema: estos corresponden a los LEDs de los semáforos vehiculares para las calles A y B, a los LEDs de los semáforos peatonales, a los botones que simulan solicitudes de cruce y a los pines que controlan un display de 7 segmentos doble en configuración de cátodo común. Luego, implementamos una rutina de inicialización que configura los pines como entradas o salidas según su función. Los pines correspondientes a los LEDs los configuramos como salidas para que después se apaguen por defecto. Los botones los configuramos como entradas con resistencias pull-up activadas, lo que nos permite detectar una pulsación activa cuando el valor de lectura es bajo. También inicializamos el display de 7 segmentos, activando sus pines y apagándolo inicialmente. Por defecto, se encienden las luces rojas de los semáforos peatonales para indicar que no se puede cruzar.

El programa utiliza funciones auxiliares que permiten cambiar el estado de los semáforos vehiculares y peatonales de forma clara y ordenada. Además, incluimos una función encargada de verificar si los botones han sido presionados. Esta verificación se hace comparando el estado actual con el estado anterior del botón para evitar que una misma pulsación se detecte varias veces. Si el botón se presiona mientras el semáforo correspondiente lo permite, es decir, cuando el semáforo vehicular está en verde, se registra una solicitud para que el peatón cruce.

También controlamos un display de 7 segmentos doble, que se usa para mostrar la cuenta regresiva durante el cruce peatonal. Una función específica descompone un número en decenas y unidades y las muestra alternadamente en cada dígito del display con una pequeña pausa para generar la ilusión de que ambos números están encendidos simultáneamente. Otra función apaga por completo el display.

Cuando un peatón tiene permitido cruzar, se ejecuta una secuencia donde se apaga temporalmente la luz roja peatonal, se enciende la luz verde y se inicia una cuenta regresiva de 10 a 0 en el display. Durante esta cuenta, la luz roja parpadea cada segundo para avisar del tiempo restante. Al terminar, se vuelve a encender la luz roja fija y se apaga la verde, indicando que ya no se debe cruzar.

El control del tráfico lo gestionamos con una función que representa una "fase". En cada fase, una calle recibe luz verde para vehículos mientras la otra se mantiene en rojo. Si hay una solicitud peatonal pendiente en la calle con semáforo rojo, se permite entonces que ese peatón cruce. Durante esta fase también monitoreamos los botones para registrar solicitudes que puedan ejecutarse en la siguiente fase. Antes de cambiar de verde a rojo, la calle activa su luz amarilla durante unos segundos.

El segundo núcleo del procesador lo utilizamos exclusivamente para revisar constantemente el estado de los botones. Esto nos garantiza que las pulsaciones no se pierdan por retrasos causados por otras tareas en el núcleo principal.

Finalmente, configuramos, en el programa principal, el sistema, inicia el segundo núcleo, y entra en un bucle infinito que alterna entre las dos fases, haciendo que el sistema funcione de manera continua y automática.

CMakeList.

Este archivo en CMake lo utilizamos para compilar un programa en la Raspberry Pi Pico. Su propósito es indicarle al sistema de compilación cómo preparar y construir el proyecto correctamente.

Primero, especificamos que se requiere como mínimo la versión 3.13 de CMake para poder usar ciertas funcionalidades. Luego, definimos el nombre del proyecto, en este caso llamado **Practica_14**.

Incluymos el archivo llamado `pico_sdk_import.cmake`, el cual es esencial porque nos permite incorporar el SDK de la Raspberry Pi Pico al proyecto. Con esto, se nos hace posible acceder a todas las funciones y herramientas que ofrece el SDK para interactuar con el hardware.

Después, inicializamos el SDK con una función específica que prepara el entorno para que el resto del script funcione correctamente.

Luego definimos el archivo principal del proyecto, llamado **Semaforo.c**, que contiene el código fuente. Este archivo es compilado en un ejecutable también llamado Semaforo.

Una vez que se definió el ejecutable, indicamos qué bibliotecas del SDK deben enlazarse con el programa. En este caso, agregamos `pico_stdlib` que ofrece funciones estándar como manejo de GPIOs y temporizadores, `hardware_adc` por si se usa el convertidor analógico-digital, y `hardware_pwm` por si se manejan señales PWM. Aunque `hardware_pwm` se incluye, su uso es opcional y dependerá del contenido de `Semaforo.c`.

Activamos la salida USB como medio de comunicación para mensajes seriales, lo que nos permite enviar información al puerto USB de la computadora desde la Pico. También desactivamos la salida por UART para evitar duplicaciones

innecesarias.

Por último, configuramos la creación del archivo .uf2, que es el formato requerido para cargar el programa final en la Raspberry Pi Pico mediante arrastrar y soltar.

0.3 Conclusión

Este proyecto me demuestra una implementación funcional y organizada de un sistema de semáforos mediante la Raspberry Pi Pico. La utilización del segundo núcleo me garantiza una respuesta inmediata ante las solicitudes peatonales, mientras que el uso de funciones auxiliares facilita la gestión clara del estado de los semáforos. Finalmente, el uso de CMake permite una compilación adecuada del proyecto, asegurandome una integración correcta con el SDK de la Pico y facilitando la generación del archivo final en formato .uf2 para su implementación práctica. A pesar de haber querido implementar dps displays de 7 segmentos multiplexados en vez de uno solo, la configuracion no se me vio benefecida, teniendo que limitarme a solo utilizar un display, sin embargo este cuenta con un funcionamiento correcto

0.4 Resultados



Figure 2: Sistema en Funcionamiento.

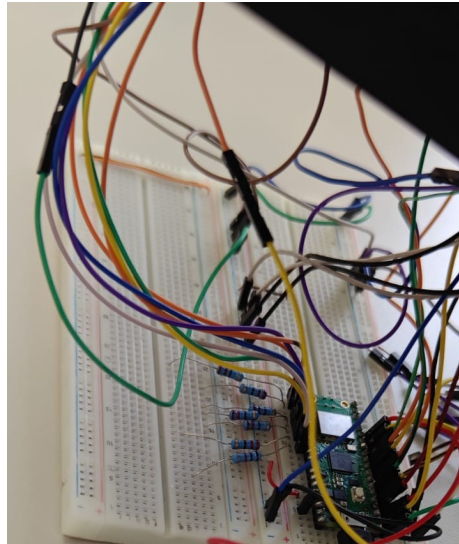


Figure 3: Cableado en Protoboard

0.5 Anexos

Código en C]

```

#include "pico/stdlib.h"
#include "pico/multicore.h"

// Pines para semáforos vehiculos A y B
#define A_RED 0
#define A_YELLOW 1
#define A_GREEN 2
#define B_RED 3
#define B_YELLOW 4
#define B_GREEN 5

// Pines para semáforos peatonales A y B
#define PED_A_RED 8
#define PED_A_GREEN 9
#define PED_B_RED 6
#define PED_B_GREEN 7

// Botones peatonales
#define BTN_PED_A 11
#define BTN_PED_B 10

// Pines de segmentos del display (A-D + DP)
const uint segment_pins[8] = {16, 17, 18, 19, 20, 21, 22, 23};

// Pines de control de dígitos del display
const uint digit_pins[2] = {24, 25};

// Dígitos para display de cátodo común
const uint8_t segment_digits[10] = {
    0b11111111, // 0
    0b00000110, // 1
    0b01101111, // 2
    0b01100111, // 3
    0b01110110, // 4
    0b01110111, // 5
    0b01111101, // 6
    0b00000111, // 7
    0b01111111, // 8
    0b01111111 // 9
};

volatile bool request_ped_A = false;
volatile bool request_ped_B = false;

// ----- Inicialización -----
void init_display() {
    for (int i = 0; i < 8; i++) {

```

Figure 4: Código en C, parte 1

[illegible]

Figure 5: Código en C, parte 2

[illegible]

Figure 6: Código en C, parte 3

```

void clear_display() {
    for (int i = 0; i < 8; i++) gpio_put(segment_pins[i], 0);
    gpio_put(digit_pins[0], 1);
    gpio_put(digit_pins[1], 1);
}

// ----- Peaton con parpadeo rojo -----
void handle_pedestrian_display(bool isA) {
    int red_pin = isA ? PWD_A_HD : PWD_B_HD;

    gpio_put(red_pin, 0);
    set_pedestrianLights(isA, 0, 1); // Apagar rojo para
    // Encender verde

    for (int i = 0; i < 10; i++) {
        gpio_put(red_pin, 1 & 2); // Parpadeo rojo
        display_number(i);
        sleep_ms(1000);
    }

    for (int i = 0; i < 3; i++) {
        gpio_put(red_pin, 0);
        clear_display();
        sleep_ms(500);
        gpio_put(red_pin, 1);
        display_number(0);
        sleep_ms(100);
    }

    clear_display();
    set_pedestrianLights(isA, 1, 0); // Apagar verde, encender
    // ----- Fin del mainloop -----

void phase1ini direction() {
    if direction {
        // Vehiculo A en verde
        set_vehicleLights(false, 1, 0, 0);
        set_vehicleLights(true, 0, 0, 1);

        if (request_ped_B) {
            request_ped_B = false;
            handle_pedestrian_display(false);
        }

        for (int i = 0; i < 100; i++) {

```

Figure 7: Código en C, parte 4

```

        check_buttons(false, true);
        sleep_ms(100);
    }

    set_vehicleLights(true, 0, 1, 0);
    sleep_ms(2000);
    set_vehicleLights(true, 1, 0, 0);

} else {
    // Vehiculo B en verde
    set_vehicleLights(true, 1, 0, 0);
    set_vehicleLights(false, 0, 0, 1);

    if (request_ped_A) {
        request_ped_A = false;
        handle_pedestrian_display(true);
    }

    for (int i = 0; i < 100; i++) {
        check_buttons(true, false);
        sleep_ms(100);
    }

    set_vehicleLights(false, 0, 1, 0);
    sleep_ms(2000);
    set_vehicleLights(false, 1, 0, 0);
}

// ----- Core secundari -----
void core1_main() {
    while (true) {
        check_buttons(true, true); // Revisar
        sleep_ms(50);
    }
}

// ----- Programa print -----
int main() {
    static_init_all();
    init_gpio();

```

Figure 8: Código en C, parte 5


```

    multicore_launch_core1(core1_main);

    bool direction = true;
    while (true) {
        phase(direction);
        direction = !direction;
    }

    return 0;
}

```

Figure 9: Código en C, parte 6

CMakeList

```

cmake_minimum_required(VERSION 3.13)

# Incluir el Raspberry Pi Pico SDK
include(pico_sdk_import.cmake)

# Inicializar el SDK
pico_sdk_init()

project(Practica_14 C CXX ASM)
set(CMAKE_C_STANDARD 11)
set(CMAKE_CXX_STANDARD 17)

add_executable(Practica_14
    semaforo.c
)

# Agregar librerías necesarias
target_link_libraries(Practica_14
    pico_stdlib
    pico_multicore # Soporte para multicore
)

# Generar binarios, uf2, etc.
pico_add_extra_outputs(Practica_14)

```

Figure 10: CMakeList